

Course Outline: Background Processing with Cronjobs

Title: Background Processing with Cronjobs: Scheduled, Reliable, Observable **Skill:** Skill 45: Background processing **Learnpack:** + Procesamiento en segundo plano con Cronjobs **File:** s45-w15d45-procesamiento-en-segundo-plano-con-cronj **Prerequisite:** s45-w15d45-tipos-de-disparadores-de-procesamiento-e **Target Audience:** Beginner developers who understand background processing principles and trigger types, now ready to implement scheduled background jobs **Total Lessons:** 9 **Format:** Mixed (11% hands-on = 1 lesson)

Course Description

A cronjob is the simplest scheduled trigger: set a time, run a function, repeat. But simple doesn't mean easy to get right. This course covers how to configure cronjobs at the system level (crontab) and at the framework level (fastapi-utilities/APScheduler), how to track their execution with timestamps and status, how to prevent overlapping runs, and how to handle failures so you can always answer "did the job run, and did it succeed?" — the minimum viability bar for any production background job.

Course Philosophy

This course emphasizes:

- Observability as the table stakes for any production cronjob
 - The overlapping problem as the most common reliability failure in scheduled jobs
 - Configuration at the right level: system crontab for OS-level tasks, framework scheduler for app-level tasks
-

Course Statistics Summary

Section	Lessons
00 - Welcome	1
01 - Cronjob Fundamentals	2
02 - Building Reliable Cronjobs	4
03 - Assessment	1
04 - Outro	1
Total	9

Learning Objectives:

- Understand that cronjobs are the implementation of cronjob-triggered background processing
- Recognize the core problems cronjobs solve: automating recurring work without human intervention
- Preview the course structure

Content Outline:

- What a cronjob does: runs a task on a schedule — nightly, hourly, weekly — without a human triggering it
- The problem it solves: some tasks must happen regularly regardless of user activity (aggregating daily stats, cleaning old records, sending digest emails)
- Contrast with user-triggered and machine-triggered jobs: cronjobs are time-driven
- The risk: a cronjob that silently fails is worse than no cronjob — nobody notices until the data is missing
- Course structure: Section 01 covers fundamentals (what cronjobs are and how to configure them); Section 02 covers reliability (observability, overlapping, failure handling, and the hands-on challenge)

Transitions: This lesson frames the problem. Section 01 covers what a cronjob is and how to configure one.

Key Principles:

- A cronjob that doesn't log its status is a black box — you'll never know if it ran
- Cronjobs are not self-healing by default: if they fail, they don't retry unless you build that in
- Configuration exists at two levels: system (crontab) and framework (APScheduler, fastapi-utilities)

Examples:

- Nightly report: every day at midnight → generate daily completion report and send to team
- Weekly digest: every Monday at 9am → email usage summary to all users
- Hourly cache refresh: every hour → recompute cached dashboard metrics

01.0 What Is a Cronjob

Learning Objectives:

- Define a cronjob as a time-triggered scheduled background job
- Read and interpret a cron schedule expression
- Explain the difference between a cron schedule and a triggered event

Content Outline:

- A cronjob is a task that runs at a specified time interval, controlled by a scheduler
- The name comes from the Unix cron daemon, but the concept applies to any scheduler
- Cron schedule expressions: the 5-field format **minute** **hour** **day-of-month** **month** **day-of-week**

- `0 9 * * 1` → 9am every Monday
- `0 * * * *` → top of every hour
- `0 0 * * *` → midnight every day
- `0 0 1 * *` → midnight on the 1st of every month
- Common shorthand: `@daily`, `@hourly`, `@weekly`
- The scheduler fires the job at the configured time regardless of any other system state
- How cron differs from event triggers: no signal needed — time itself is the trigger
- The cron daemon vs. framework schedulers: cron is a system-level daemon; APScheduler/fastapi-utilities are application-level schedulers — same concept, different scope
- Section preview: 01.1 shows how to configure cronjobs at both levels

Transitions: This lesson explains what a cronjob is. 01.1 shows how to set one up.

Key Principles:

- The cron expression is the trigger — it encodes "when" the job runs
- Cronjobs are stateless by default: each run is independent; the scheduler doesn't know if the previous run succeeded
- Always test cron expressions with a validator before deploying to production

Examples:

- `30 8 * * 1-5` → 8:30am Monday through Friday
- `*/15 * * * *` → every 15 minutes
- `0 0 * * 0` → midnight every Sunday

01.1 Setting Up Cronjobs

Learning Objectives:

- Configure a cronjob using the system crontab
- Configure a scheduled background job using APScheduler in a FastAPI application
- Choose between system-level and framework-level scheduling based on context

Content Outline:

- **System-level: crontab**
 - Access: `crontab -e` opens the crontab file in the default editor
 - Format: `minute hour dom month dow command`
 - Example: `0 9 * * 1 /usr/bin/python3 /app/scripts/weekly_report.py`
 - The cron daemon runs the command as a subprocess at the scheduled time
 - Environment: crontab runs in a minimal shell environment — always use absolute paths; don't rely on user environment variables
 - Listing and removing: `crontab -l` to list, `crontab -r` to remove all
 - Best for: system-level tasks, scripts that run outside a web app, OS maintenance tasks
- **Framework-level: fastapi-utilities / APScheduler**
 - Purpose: schedule Python functions to run within your FastAPI application process
 - Library: `apscheduler` (AsyncIOScheduler) or `fastapi-utils` (`repeat_every`)

- Basic APScheduler setup:

```
from apscheduler.schedulers.asyncio import AsyncIOScheduler
scheduler = AsyncIOScheduler()

@scheduler.scheduled_job('cron', hour=0, minute=0)
async def nightly_job():
    # runs at midnight every day
    pass

scheduler.start() # start in FastAPI startup event
```

- The scheduler runs inside your FastAPI process, so it starts with the app and stops when the app stops
- Best for: app-level tasks that need access to your app's database connections, configuration, and services
- **Choosing between crontab and APScheduler:**
 - crontab: OS-independent, survives app restarts, good for scripts; no access to app state
 - APScheduler: tight integration with app, can use same DB connections; stops if app stops

Transitions: You can now configure a cronjob. Section 02 covers making it reliable — observable, non-overlapping, and failure-aware.

Key Principles:

- System crontab and app-level schedulers are both valid — choose based on whether the task needs access to app state
- Always use absolute paths in crontab
- APScheduler starts inside the app process — if the app restarts frequently, jobs may be delayed or missed

Examples:

```
# APScheduler in FastAPI
from fastapi import FastAPI
from apscheduler.schedulers.asyncio import AsyncIOScheduler

app = FastAPI()
scheduler = AsyncIOScheduler()

@scheduler.scheduled_job('cron', hour=9, minute=0, day_of_week='mon')
async def send_weekly_digest():
    # sends weekly email digest every Monday at 9am
    pass

@app.on_event("startup")
async def startup():
    scheduler.start()
```

```
@app.on_event("shutdown")
async def shutdown():
    scheduler.shutdown()
```

02.0 Building Reliable Cronjobs

Learning Objectives:

- Name the three components of a reliable cronjob: observability, overlap prevention, and failure handling
- Understand why a "working" cronjob is not the same as a "reliable" cronjob
- Preview what each subsequent lesson covers

Content Outline:

- The gap between working and reliable:
 - A working cronjob runs at the scheduled time
 - A reliable cronjob runs at the scheduled time AND you can always answer: "Did it run? Did it succeed? What did it do?"
- Three reliability dimensions:
 1. **Observability:** track execution timestamps, status, and output — "poder saber qué pasó"
 2. **Overlap prevention:** ensure a new run doesn't start while the previous one is still running
 3. **Failure handling:** when a run fails, capture the error, decide whether to retry, and log enough context to investigate
- Why all three are needed:
 - Without observability: failures are silent; you find out from users
 - Without overlap prevention: data can be double-processed or corrupted
 - Without failure handling: a single bad run stops all future runs (in some frameworks) or silently skips them
- Section preview: 02.1 covers observability and status tracking; 02.2 covers overlapping and failure handling; 02.3 is the hands-on challenge

Transitions: This overview sets up the next three lessons. 02.1 starts with observability.

Key Principles:

- "The job ran" and "the job succeeded" are different facts — you need to track both
- Reliability is not free — it requires deliberate logging, locking, and error handling
- The three dimensions are interdependent: failure handling without observability still leaves you blind

Examples:

- A cronjob that runs daily but has never had its success confirmed: you only know it failed when users complain

- A cronjob that generates overlapping runs on a slow day: the same records are processed twice, creating duplicate reports
 - A cronjob that fails and swallows the exception: it appears to "succeed" (no error) but produces no output
-

02.1 Observability and Status 📖

Learning Objectives:

- Log a structured record for every cronjob execution: `started_at`, `status`, and `error`
- Assign status values to data chunks being processed to track progress
- Explain why assigning status to data is a best practice for background jobs

Content Outline:

- **Execution-level observability:**
 - Minimum log entry per run: `{job: "job_name", started_at: ISO_timestamp, status: "running|completed|failed", error: null|"message"}`
 - Log at the start (`status: "running"`) and at the end (`status: "completed"` or `"failed"`)
 - Why start logging: if the job crashes before the end log, you know it was running at `start_at` but never completed
 - Structured logs (JSON) are searchable and can be ingested by monitoring tools
- **Data-chunk level observability:**
 - From the curriculum: "Background processing is better done if we assign a status to the data chunk that is being processed"
 - What this means: if you're processing records in batches, each record should have a status column (`pending` → `processing` → `done` | `failed`)
 - Why: if the job crashes mid-batch, you can resume from where it stopped by querying `WHERE status = 'pending'`
 - Example: processing telemetry events → each event gets `status = "processing"` before processing, `status = "done"` after
 - Anti-pattern: loading all records into memory and processing them without any status tracking — one crash means starting over
- Connecting to the background processing principles (from the intro learnpack): this is observability + durability in practice

Transitions: You now know how to make a job's execution visible. 02.2 covers what to do when two runs overlap or when a run fails.

Key Principles:

- Log before you act: the start log exists precisely for cases where the job crashes before it finishes
- Data-chunk status is your crash recovery mechanism — it lets you resume rather than restart
- "Status = processing" during processing, then "status = done" — never skip the intermediate state

Examples:

```

import json
from datetime import datetime

async def generate_daily_report():
    log = {"job": "daily_report", "started_at": datetime.now().isoformat(),
"status": "running"}
    print(json.dumps(log))
    try:
        # process data
        count = process_records()
        log.update({"status": "completed", "records_processed": count})
    except Exception as e:
        log.update({"status": "failed", "error": str(e)})
    finally:
        log["finished_at"] = datetime.now().isoformat()
        print(json.dumps(log))

```

02.2 Overlapping and Failure Handling 📖

Learning Objectives:

- Define the overlapping cronjob problem and explain when it occurs
- Implement a simple locking mechanism to prevent concurrent runs
- Apply error handling patterns that capture failures without silencing them

Content Outline:

- **The Overlapping Problem:**
 - Occurs when: a new scheduled run starts while the previous run is still executing
 - Why it happens: the scheduler fires at the configured time regardless of whether the previous run has finished
 - Concrete example: a report job takes 10 minutes; it's scheduled every 5 minutes → two instances run concurrently → the report is generated twice from the same data → duplicate records
 - Detection: if you have execution-level observability, you can see two "running" entries with no "completed" between them
 - Prevention strategies:
 1. **Process-level locking:** check if a flag/lock file exists before starting; create it at start and delete at end
 2. **Database-level locking:** check a `job_running` row/flag in the database before starting
 3. **APScheduler `max_instances=1`:** APScheduler's built-in parameter to prevent concurrent runs of the same job
 4. **Longer interval than execution time:** simplest; if the job takes 5 minutes maximum, schedule it every 10 minutes
 - Best practice: set a lock timeout — if the lock is older than 2x the expected runtime, assume the previous run crashed and clear it

- **Failure Handling:**

- Gestión de fallos: poder saber lo que pasó (failure management: being able to know what happened)
- Cronjobs fail for many reasons: external service down, database unavailable, data format changed, code bug
- The failure handling checklist:
 1. **Catch exceptions explicitly:** never let a cronjob fail silently
 2. **Log the exception and context:** job name, timestamp, error message, stack trace if needed
 3. **Update status to "failed":** so observability shows the failed run
 4. **Decide: retry or alert?:** automatic retry on transient failures (network); alert on persistent failures (bad data, code bug)
- Woven-in best practice: do NOT raise the exception after logging it if you don't want the scheduler to stop scheduling future runs — handle it gracefully
- Common failure modes:
 - Transient (retry): database timeout, external service unavailable, network error
 - Permanent (alert + investigate): data validation error, configuration missing, code exception

Transitions: You can now build cronjobs that are observable, non-overlapping, and failure-aware. 02.3 brings everything together in a hands-on challenge.

Key Principles:

- The overlapping problem is the most common reliability failure for scheduled jobs — always implement prevention
- A caught exception that is only logged (not re-raised) allows the scheduler to keep running future jobs
- The distinction between transient and permanent failures determines whether to retry or alert

Examples:

```
# APScheduler overlap prevention
@scheduler.scheduled_job('cron', hour=0, minute=0, max_instances=1)
async def daily_report():
    # APScheduler won't start a new instance if one is already running
    pass

# Manual locking (for crontab or other systems)
import fcntl
import sys

try:
    lock_file = open('/tmp/daily_report.lock', 'w')
    fcntl.flock(lock_file, fcntl.LOCK_EX | fcntl.LOCK_NB)
except IOError:
    print("Job already running. Exiting.")
    sys.exit(0)
```



```
# ... run job ...  
fcntl.flock(lock_file, fcntl.LOCK_UN)
```

02.3 Schedule a Report Job

Challenge Prompt: Write a FastAPI application that uses APScheduler to schedule a daily background job at midnight. The job should load a CSV file of TODO events, filter for completions from the last 24 hours, count them, and print a structured JSON log entry with the job name, start time, status, and count (or error message on failure).

Solution Code:

```
from fastapi import FastAPI  
from apscheduler.schedulers.asyncio import AsyncIOScheduler  
import pandas as pd  
from datetime import datetime, timedelta  
import json  
  
app = FastAPI()  
scheduler = AsyncIOScheduler()  
  
@scheduler.scheduled_job('cron', hour=0, minute=0, max_instances=1)  
async def generate_daily_report():  
    log = {  
        "job": "daily_report",  
        "started_at": datetime.now().isoformat(),  
        "status": "running"  
    }  
    print(json.dumps(log))  
    try:  
        df = pd.read_csv('todos.csv')  
        df['completed_at'] = pd.to_datetime(df['completed_at'])  
        cutoff = datetime.now() - timedelta(days=1)  
        completed = df[  
            (df['status'] == 'completed') &  
            (df['completed_at'] >= cutoff)  
        ]  
        count = len(completed)  
        log.update({"status": "completed", "count": count,  
                   "finished_at": datetime.now().isoformat()})  
    except Exception as e:  
        log.update({"status": "failed", "error": str(e),  
                   "finished_at": datetime.now().isoformat()})  
    print(json.dumps(log))  
  
@app.on_event("startup")  
async def startup():  
    scheduler.start()
```

```
@app.on_event("shutdown")
async def shutdown():
    scheduler.shutdown()
```

Challenge Code:

```
from fastapi import FastAPI
from apscheduler.schedulers.asyncio import AsyncIOScheduler
import pandas as pd
from datetime import datetime, timedelta
import json

app = FastAPI()
scheduler = AsyncIOScheduler()

# Define a scheduled job that runs daily at midnight (max_instances=1)
# The job should:
# 1. Log start: {"job": "daily_report", "started_at": ..., "status": "running"}
# 2. Load 'todos.csv' into a pandas DataFrame
# 3. Convert 'completed_at' to datetime
# 4. Filter: status == 'completed' AND completed_at >= 24 hours ago
# 5. Count matching rows
# 6. On success: log {"status": "completed", "count": ..., "finished_at": ...}
# 7. On failure: log {"status": "failed", "error": ..., "finished_at": ...}
# your code here

@app.on_event("startup")
async def startup():
    # your code here – start the scheduler
    pass

@app.on_event("shutdown")
async def shutdown():
    # your code here – shutdown the scheduler
    pass
```

03.0 Assessment 🧐

Learning Objectives:

- Apply knowledge of cronjob configuration, observability, and reliability to scenario questions
- Diagnose common cronjob failures and identify the appropriate fix
- Choose between crontab and APScheduler for a given context

Question Format: Scenario-based (multiple-choice)

Questions:

1. A cronjob runs every 5 minutes to aggregate telemetry data. The aggregation takes 8 minutes. What will happen?

- A) The scheduler will skip the next run since the previous one hasn't finished
- B) Two runs will execute concurrently, potentially processing the same data twice ✓
- C) The job will fail with a timeout error
- D) APScheduler will automatically queue the next run until the current one finishes

2. Your nightly report cronjob runs at midnight but nobody knows if it succeeds or fails. Users complain about missing reports on Monday. What was missing?

- A) A retry mechanism
- B) A lock file
- C) Execution logging with status tracking ✓
- D) Longer job interval

3. Which APScheduler parameter prevents a second instance of a job from starting while the first is still running?

- A) `timeout=True`
- B) `max_instances=1` ✓
- C) `singleton=True`
- D) `no_overlap=True`

4. You want to assign a status to each telemetry record being processed by your cronjob. What is the correct status sequence?

- A) `queued → done → failed`
- B) `pending → processing → done | failed` ✓
- C) `new → active → complete`
- D) `created → running → archived`

5. Your cronjob catches an exception and prints a log entry. You do NOT re-raise the exception. What happens to future scheduled runs?

- A) The scheduler stops scheduling this job permanently
- B) The scheduler marks this job as failed and stops future runs
- C) Future runs continue to be scheduled and execute ✓
- D) The scheduler retries the current run immediately

6. You need to schedule a Python script that cleans old log files. The script runs independently of your web app and needs to work even when the app is down. Which option is best?

- A) APScheduler in your FastAPI app
- B) System crontab ✓
- C) A scheduled endpoint called by the frontend
- D) A background thread in the app server

7. Your cronjob job log entry shows `status: "running"` from 3 days ago with no subsequent `status: "completed"` or `status: "failed"` entry. What most likely happened?
- A) The job is still running successfully
 - B) The log entry was corrupted
 - C) The job crashed before it could write the final status ✓
 - D) The scheduler stopped all jobs
8. Which of the following is a transient failure that should trigger a retry?
- A) A JSON parsing error in the input data
 - B) A missing required configuration key
 - C) A database connection timeout ✓
 - D) A division-by-zero error in the aggregation code

Evaluation Criteria:

- Correctly identifies the overlapping problem and its cause
- Understands the role of structured logging in observability
- Applies the right APScheduler configuration for reliability
- Distinguishes transient from permanent failures

Score Interpretation:

- 8/8: You can build production-ready cronjob pipelines
 - 6–7: Review any missed lessons before moving on
 - 4–5: Re-read Section 02 with focus on reliability patterns
 - Below 4: Restart from Section 01
-

04.0 Scheduled Automation

Content Outline:

- Course recap: what students accomplished
 - Learned what a cronjob is and how to read a cron schedule expression
 - Configured cronjobs at the system level (crontab) and app level (APScheduler)
 - Built a reliable cronjob with execution logging, data-chunk status tracking, overlap prevention, and failure handling
 - Wrote a complete FastAPI + APScheduler application that runs a report job on a schedule
- Connection to bigger picture: every production service that runs scheduled work — report generation, cache refresh, billing, data pipelines — uses these patterns. The specifics of the scheduler tool change; the requirements (log it, don't overlap it, handle failures) don't.
- Next steps: Skill 46 covers queue management (the foundational data structure behind user-triggered, third-party, and chained processing); Skill 47 covers async processing with message brokers — together they complete the background processing toolkit
- Resources for further exploration:
 - APScheduler documentation: scheduler types, job stores, executors
 - Crontab.guru: interactive cron expression editor and validator

- FastAPI official docs: background tasks (note: for lightweight cases, FastAPI's `BackgroundTasks` is simpler than `APScheduler` but has no scheduling)
- Encouragement: the hardest part about production cronjobs is not the scheduling — it's the discipline to always log, always track status, always prevent overlaps. Once those habits are in place, every scheduled job you build will be reliable by default.

Note: This is conclusion only — no theory, exercises, or assessment.